TradingAlgorithms_HW5_Documentation

برای زیاد شدن داک عذرخواهی میکنم. سعی کردم تمام موارد رو پوشش بدم و جاهایی رو هم کمتر توضیح دادم ولی به خاطر عکس ها و تکه کدها صفحات زیاد شد وگرنه خود مطالب اینقدر جا نمیگرفتند.
باز پوزش میخواهم.

برای تکه کدهای طولانی "Read the full in notebook ..." گذاشتم تا داک کمتر شه و کد کامل در notebook قابل مشاهده هست.

نکاتی قبل از شروع داک

- من استراتژی رو روی هر تایم فریمی از کوین ها که شرط رو برقرار میکردند تست کردم یعنی مثلا XMR در تایم فریم یک روز فقط شرط را دارد و در دو تایم فریم دیگر این شرط رو ندارند پس من فقط در تایم فریم یک روزه انجام دادم. از TA پرسیدم گفتند که کاملا اوکیه و فقط اینجا توی داک هم بنویسم.
- یک مشکلی اساسی که اینجا خوردم هیچ کوینی در طول سال کاملا بالای ema 200 نبود.(منطقا هم کل سال احتمالش خیلی کمه). برای همین یک درصد threshold قرار دادم که مثلا اگر در طول سال ۷۰ درصد بالا ema باشه اون رو قبول کنیم.
- وقتی معاملات کم هستند، سالانه‌سازی برای شارپ نتایج بزرگی نشون می‌دهد. به خصوص اینو توی نتایج میبینیم. چون تعداد ترید ها کمه یکی دو ترید در ماه انجام میدیم به خاطر همین *STD* خیلی کم میشود و مقدار شارپ بزرگ میشود و این به دلیل بالا میباشد. گفتم خوب باشه اینو همین اول توضیح بدم.(در پایین تر هم گفتم)

بخش صفر

در این بخش قبل از خود استراتژی ها باید بریم و دیتاهای برترین ۵۰ کوین با بیشترین مارکت کپ رو بدست بیاریم.

یک نکته برای این بخش:

> پنجاه کوین برتر همگی در یاهو فایننس و بایننس نبودند و نمیشد که دانلود کرد. من یه اپشن استفاده از کوین هاب بعد پنجاه تا برتر هم گذاشتم ولی ازش استفاده نکردم تا پیش فرض سوال عوض نشه. ولی خب گفتم که به این موضوع حواسم بوده و یه اپشنی گذاشتم.

> یک نکته‌ای که اینجا هست موارد اخر این لیست (۴۰ تا ۵۰) ثابت نیستند و هرروز ممکنه عوض بشن و من روزی که لیست رو دراوردم این بود

لیست موقع که من دراوردم این موارد بودند:

```
COINS = [
    ("BTC","Bitcoin"), ("ETH","Ethereum"), ("USDT","Tether"),
("BNB","BNB"),
    ("XRP","XRP"), ("USDC","USDC"), ("SOL","Solana"), ("TRX","TRON"),
    ("DOGE","Dogecoin"), ("ADA","Cardano"), ("BCH","Bitcoin Cash"),
("HYPE","Hyperliquid"),
    ("LINK","Chainlink"), ("LEO","UNUS SED LEO"), ("XMR","Monero"),
```

```
Read the full in notebook...
]
```

> ✎ پنجاه کوین برتر همگی در یاهو فایننس و بایننس نبودند و نمیشد که دانلود کرد. من یه اپشن استفاده ازکوین های بعد پنجاه تا برتر هم گذاشتم ولی از ش استفاده نکردم تا پیش فرض سوال عوض نشه. ولی خب گفتم که به این موضوع حواسم بوده و یه اپشنی گذاشتم.

از بین این کوین ها موارد زیر یا کلا نبودند یا در تاریخ ما و در تایم فریم‌های ما کامل نبودند.

```
[-] HYPE (ALL): 1d: No data from any source
[-] USDe (ALL): 1d: insufficient data (83 rows)
[-] MNT (ALL): 1d: No data from any source
[-] WLFI (ALL): 1d: insufficient data (91 rows)
[-] USD1 (ALL): 1d: insufficient data (193 rows)
[-] CC (ALL): 1d: No data from any source
[-] ASTER (ALL): 1d: insufficient data (56 rows)
[-] PI (ALL): 1d: insufficient data (100 rows)
[-] USDG (ALL): 1h: insufficient data (7609 rows)
```

و دیتاها در فولدر **Data/** ذخیره شده‌اند.

کد دانلود کردن کوین‌ها:

```
import time
OUT_DIR = os.path.join(os.getcwd(), DATA_SAVE_DIR)
os.makedirs(OUT_DIR, exist_ok=True)

Read the full in notebook...
```

برای دانلود داده‌های قیمتی کوین‌ها از دو منبع Yahoo Finance و Binance استفاده کردیم:

- ۱. `yahoo_ticker_for(symbol)` : نماد کوین را به فرمت تیکر Yahoo Finance (`SYMBOL-USD`) تبدیل می‌کند
- ۲. `download_from_yahoo(symbol)` : داده‌های OHLCV را از Yahoo Finance دریافت می‌کند. در صورت عدم موفقیت، تیکر `SYMBOL-USDT` را هم امتحان می‌کنیم
- ۳. `download_from_binance(symbol)` : برای دریافت داده‌های کندل استیک روزانه از Binance API استفاده می‌کند. به عنوان منبع دوم در صورت نداشتن کوین در Yahoo Finance
- ۴. `save_df_csv(symbol, df)` : دیتافریم را در فایل CSV با فرمت `SYMBOL_1D.csv` ذخیره می‌کند
- ۵. `download_coins()` : فانکشن اصلی که برای هر کوین ابتدا در Yahoo Finance را امتحان می‌کند و در صورت وجود نداشتن یا داده ناکافی، از Binance استفاده می‌کند

بخش اول و دوم

در این بخش میخواهیم کوین هایی که شرایط سوال را دارند پیدا کنیم.

یک مشکلی اساسی اینجا خوردم هیچ کوینی در طول سال کلا بالای ema 200 نبود.(منطقا هم کل سال احتمالش خیلی کمه). برای همین یک درصد ترشهولد قرار دادم که مثلا اگر در طول سال ۷۰ درصد بالا ema باشه اون رو قبول کنیم

```
def generate_buy_grid_levels(current_price, num_levels, step_percent):
    buy_levels = []


Read the full in Notebook...
```

در این فانکشن ما سطح های گرید رو براساس تعداد و فاصله‌شون درست میکنیم.

```
def check_price_channel(symbol, timeframe='1d', ema_period=EMA ,
threshold_pct=EMA_ABOVE_THRESHOLD):
    interval_label = timeframe.upper()

Read the full in Notebook...
```

در فانکشن بالا دقیقا شرایط سوال رو چک میکنیم که ایا قیمت کوین در ۷۰ درصد بازه بالای ema 200 بوده یا نه.

```
def check_price_channel_all_timeframes(symbol, ema_period=EMA,
threshold_pct=EMA_ABOVE_THRESHOLD):

Read the full in Notebook...
```

در فانکشن بالا چون ما سه تایم فریم مختلف داریم من هر کوین رو در هر سه تست میکنم و اونایی که در تایم فریمی قبول شدند فقط در همون تایم فریم استراتژی رو اجرا میکنم.

```
def calculate_atr_and_grid(df, period=ATR_PERIOD,
atr_multiplier=GRID_ATR_MULTIPLIER):
    high_low = df['High'] - df['Low']
    high_close_prev = abs(df['High'] - df['Close'].shift(1))

Read the full in notebook...
```

در فانکشن بالا همان طور که گفته شده بود، ما فاصله گرید رو براساس *ATR* حساب میکنیم و فرمول فاصله

grid_spacing = current_atr * atr_multiplier

و در فرمول یک ضریب ثابت به عنوان کانستنت درنظر میگیریم.

```
def calculate_average_purchase_price(buy_levels, volumes):
    if len(buy_levels) == 0:
        return 0.0

Read the full in notebook...
```

در فانکشن بالا میانگین قیمت خرید رو براساس قیمت های مختلف و حجم هر خرید محاسبه میکنیم. این میانگین وزن‌دار هست و برای تعیین نقطه سود و ضرر استفاده میشه.

```
def calculate_martingale_size(base_volume, multiplier =
MARTINGALE_MULTIPLIER, level = 1):

Read the full in notebook...
```

در این فانکشن حجم خرید در هر سطح از گرید رو براساس استراتژی مارتینگل محاسبه میکنیم. با هر سطح پایین‌تر، حجم خرید به اندازه ضریب مارتینگل افزایش پیدا میکنه.

بخش سوم

✎ کاری که من برای پیاده سازی انجام دادم این بود که یک استیت کلی درست کنم و در طول استراتژی با این استیت کار میکنیم. با این کار، کار کردن و درآوردن متریک ها خیلی راحت تر شدند.

```
state = {
    'initial_capital': initial_capital,
    'capital': initial_capital,
    'base_volume': base_volume,
    'max_levels': max_levels,
    'grid_spacing_pct': grid_spacing_pct,
    'tp_percent': tp_percent,
    'sl_level': sl_level,
    'martingale_multiplier': martingale_multiplier,
    'grid_levels': [],
    'positions': [],
    'grid_active': False,
    'stop_trading': False,
    'trades': [],
    'total_profit': 0.0,
    'total_trades': 0,
    'winning_trades': 0,
    'losing_trades': 0
}
```

```
def get_position_volume_for_level(level, base_volume,
martingale_multiplier):
```

```
    return calculate_martingale_size(base_volume, martingale_multiplier,
level)
```

در فانکشن بالا صرفا فانکشنی که برای سایز دربالاتر درست کرده بودیم رو صدا میکنیم و یک ابسترکشن برای اون هست.

```
def calculate_total_volume(positions):
    return sum(volume for _, volume in positions)
```

در فانکشن بالا هم مجموع کل حجم رو بدست میاریم.

```
def check_take_profit_condition(current_price, positions, tp_percent):
    if not positions:
        return False

Read the full in Notebook...
```

در این فانکشن شرط رسیدن به هدف سود را بررسی میکنیم. ابتدا اگر پوزیشنی باز نیست برمی‌گردیم `False`. در غیر این صورت:

- میانگین قیمت ورود را با میانگین‌گیری محاسبه می‌کنیم
- قیمت هدف را با اضافه کردن درصد `tp_percent` به میانگین قیمت تعیین می‌کنیم
- اگر قیمت فعلی از قیمت هدف بیشتر باشد، شرط برقرار است و `True` برمی‌گردانیم

```
def check_stop_loss_condition(current_price, grid_levels, sl_level):
    if len(grid_levels) ≤ sl_level:
        return False

Read the full in Notebook...
```

در این فانکشن شرط استاپ‌لس را بررسی میکنیم. اگر `sl_level` بیشتر از تعداد سطوح موجود باشد (خارج از بازه)، `False` برمی‌گردد. در غیر این صورت:

- قیمت Stop Loss را از سطح مشخص شده در گرید می‌گیریم
- اگر قیمت فعلی از این سطح کمتر یا برابر باشد، استاپ‌لس فعال میکنیم

```
def execute_buy_order(state, price, level):
    volume = get_position_volume_for_level(level, state['base_volume'],
state['martingale_multiplier'])
    cost = price * volume

Read the full in Notebook...
```

در این فانکشن یک سفارش خرید شبیه سازی میکنیم:

- حجم مورد نیاز برای سطح مشخص شده را محاسبه می‌کنیم
- هزینه کل خرید را با ضرب قیمت در حجم بدست می‌آوریم
- اگر هزینه از سرمایه موجود بیشتر باشد، سفارش ناموفق است
- در غیر این صورت:
  - سرمایه را کم می‌کنیم
  - پوزیشن جدید را به لیست پوزیشن‌های باز اضافه می‌کنیم (قیمت و حجم)

```
def close_all_positions(state, current_price, reason="TP"):
    if not state['positions']:
        return 0

    total_volume = calculate_total_volume(state['positions'])
    prices = [p[0] for p in state['positions']]


Read the full in Notebook...
```

در این فانکشن تمامی پوزیشن‌های باز را می‌بندیم و سود/زیان رو ثبت می‌کنیم:

- اگر پوزیشن باز نیست، `0` برمی‌گردد
- در غیر این صورت:
  - کل حجم باز را محاسبه می‌کنیم
  - میانگین قیمت ورود و کل هزینه را بدست می‌آوریم
  - درآمد از فروش را محاسبه کرده و سود/زیان را تعیین می‌کنیم
  - سرمایه را افزایش می‌دهیم
  - معامله را به `state['trades']` اضافه می‌کنیم با تمام جزئیات
  - متریک‌های کل را به‌روزرسانی می‌کنیم (سود کل، تعداد معاملات، تعداد معاملات برنده/بازنده)
  - گرید را ریست می‌کنیم
  - اگر دلیل `SL` باشد (استاپ‌لس)، متوقف میکنیم

```
def backtest_grid_strategy(df, state, debug=False):
    results = []
    grid_initialized = False

    for idx, row in df.iterrows():
        if state['stop_trading']:
            break

Read the full in Notebook...
```

این حلقه اصلی بک‌تست برای استراتژی Martingale Grid است. مراحل کار:

- برای هر ردیف کندل، قیمت بستن را می‌گیریم

- در اول (هنگام اولین ورود):
- و فاصله گرید را محاسبه می‌کنیم ATR
  - سطوح گرید را تولید می‌کنیم
  - گرید را فعال می‌کنیم
- در هر قدم:
  - بررسی می‌کنیم که آیا قیمت به یکی از سطح‌های خرید رسیده
  - اگر رسیده و هنوز پوزیشن در آن سطح باز نیست، خرید اجرا می‌کنیم
  - شرط TP را بررسی می‌کنیم. اگر برقرار بود تمامی پوزیشن‌ها بسته می‌شوند
  - شرط SL را بررسی می‌کنیم. اگر برقرار بود تمامی پوزیشن‌ها بسته و تجارت متوقف می‌شود
  - اگر معامله ای بسته شده بود، گرید دوباره مقداردهی میکنیم

نحوه تست ماهانه این بخش:

```
def test_grid_strategy_monthly(symbol, year=2025, timeframe='1d',
initial_capital=INITIAL_CAPITAL, base_volume=BASE_VOLUME, debug=False):
    try:
        interval_label = timeframe.upper()

Read the full in Notebook...
```

✎ من فاصله گریدهارو هر ماه با محاسبه دوباره *ATR* به دست میارم

در این فانکشن برای هر ماه از سال، استراتژی گرید را تست میکنیم و نتایج هر ماه را جداگانه ثبت میکنیم. برای هر ماه:

- داده‌های آن ماه را از سال مشخص شده تفکیک میکنیم
- و فاصله گرید را محاسبه میکنیم ATR
- استراتژی را اجرا میکنیم و تمام متریک‌های مهم (تعداد معاملات، سود، Sharpe Ratio و غیره) را جمع‌آوری میکنیم
- نتایج در یک لیست برای نمایش بعدی ذخیره میشود

```
def test_all_timeframes(symbol, year=2025,
initial_capital=INITIAL_CAPITAL):
    timeframes = ['1h', '4h', '1d']
    all_results = {}

 Read the full in Notebook...
```

در این فانکشن استراتژی را برای یک کوین در هر سه تایم فریم (۱H، ۴H، ۱D) تست میکنیم و نتایج هر کدام رو جداگانه ذخیره میکنیم.

```
def print_monthly_results(monthly_results, symbol, timeframe):
    print(f"monthkly result for {symbol} - {timeframe} timeframe")

Read the full in Notebook...
```

در این فانکشن نتایج ماهانه را به صورت جدول نمایش میدهیم. برای هر ماه متریک های مهم مثل تعداد معاملات، درصد برنده، سود و Sharpe Ratio چاپ میشود و در انتها میانگین تمام ماه‌ها نمایش داده میشود.

```
def print_timeframe_summary(all_results, symbol):

Read the full in Notebook...
```

در این فانکشن نتایج نهایی را برای هر تایم فریم مقایسه میکنیم. میانگین متریک های (تعداد معاملات، سود و غیره) را برای هر تایم فریم محاسبه کرده و در یک جدول نمایش میدهیم تا بتوانیم کدام تایم فریم بهتر عمل کرده است مشخص کنیم.

بخش چهارم

فانکشن های این بخش تقریبا همان بخش دو هست فقط با منطق *Anti Martingle*.

✎ همانند بخش دوم من اینجا یک استیت کلی برای استراتژی درنظر گرفتم

```
def create_anti_martingale_grid_levels(current_price, grid_spacing_pct,
max_levels):
    grid_levels = [current_price]

Read the full in Notebook...
```

در این فانکشن سطوح گرید را برای استراتژی Anti-Martingale درست میکنیم. قیمت فعلی نقطه ورود درنظر میگیریم و سطح‌های بعدی بالاتر از آن با درصد فاصله مشخص شده قرار میگیرند (برای خریدهای اضافی هنگام افزایش قیمت).

```
def check_anti_martingale_stop_loss(current_price, entry_price,
sl_percent):
    sl_price = entry_price * (1 - sl_percent / 100)
    return current_price ≤ sl_price
```

در این فانکشن بررسی میکنیم که آیا قیمت به استاپ‌لس رسیده یا نه. قیمت Stop Loss را با کاهش درصد مشخص شده از قیمت ورود محاسبه میکنیم و اگر قیمت فعلی کمتر یا برابر آن باشد، شرط برقرار است.

```
def check_anti_martingale_take_profit(current_price, positions,
tp_percent):
    if not positions:
```

```
        return False

Read the full in Notebook...
```

در این فانکشن شرط رسیدن به *take profit* را بررسی میکنیم. اگر پوزیشنی باز نیست `False` برمی‌گردد. در غیر این صورت میانگین قیمت ورود را محاسبه کرده و قیمت هدف را با اضافه کردن درصد `tp_percent` تعیین میکنیم. اگر قیمت فعلی بیشتر یا برابر قیمت هدف باشد، شرط برقرار است.

```
def execute_anti_martingale_buy(state, price):
    volume = state['current_volume']
    cost = price * volume

Read the full in Notebook...
```

در این فانکشن یک سفارش خرید را اجرا میکنیم. حجم خرید را از state میگیریم و اگر پول کافی نباشد، تمام پول باقی‌مانده را برای خرید استفاده میکنیم. سپس پوزیشن جدید را به لیست اضافه کرده و سرمایه را کاهش میدهیم.

```
def close_anti_martingale_positions(state, current_price, reason="TP"):
    if not state['positions']:
        return 0

Read the full in Notebook...
```

در این فانکشن تمام پوزیشن‌های باز را می‌بندیم و سود/زیان را ثبت میکنیم. تفاوت اصلی با Martingale این است که اگر معامله برنده بود، حجم خرید بعدی را افزایش میدهیم (Anti-Martingale)، اما اگر بازنده بود، به حجم پایه برمی‌گردیم. سپس گرید را ریست میکنیم برای معاملات بعدی.

```
def backtest_anti_martingale_grid(df, state):
    results = []

Read the full in Notebook...
```

این حلقه اصلی بک‌تست برای استراتژی Anti-Martingale Grid است. تفاوت اصلی با Martingale :

- در ابتدا سطوح گرید را بالای قیمت فعلی درست میکنیم (نه پایین)
- فقط زمانی خریدهای اضافی انجام میدهیم که قیمت بالا رود (یعنی درحال سودده باشیم)
- اگر معامله برنده بود، حجم بعدی افزایش مییابد (افزایش در پروفیت)
- اگر معامله بازنده بود، حجم به حالت اول برمی‌گردد

نحوه تست ماهانه این بخش:

فانکشن های این بخش خیلی شبیه بخش دو بودند و فقط از فانکشن های جدیدمون استفاده میکنیم. برای زیاد نشدن داک این کدهارو نیاوردم.

بخش پنجم

متریک ها

```
def calculate_sharpe_ratio(trades, risk_free_rate=RISK_FREE_RATE,
annualize=True):
    if not trades or len(trades) < 2:
        return 0.0
   Read the full in Notebook...
```

- `annualize=True` یعنی نتایج سالانه‌شده برگردانیم
- اگر Sharpe : `annualize=False` ماهانه حساب میشود (بدون ضرب کردن)
- اگر `annualize=True` : `trades_per_year = num_trades * 12` (فرض میکنیم ۱۲ ماه است)
- بازدهی متوسط را در `trades_per_year` ضرب میکنیم
- انحراف معیار را در `sqrt(trades_per_year)` ضرب میکنیم
- نتیجه: Sharpe = (بازدهی سالانه - نرخ بدون ریسک) / ریسک سالانه

> وقتی معاملات کم هستند، سالانه‌سازی نتایج بزرگ‌تری می‌دهد. به خصوص اینو جلوتر میبینیم. چون تعداد ترید ها کمه یکی دو ترید در ماه انجام میدیم به خاطر همین *STD* خیلی کم میشود و مقدار شارپ بزرگ میشود

```
def calculate_sortino_ratio(trades, risk_free_rate=RISK_FREE_RATE,
target_return=0.0, annualize=True):
    if not trades or len(trades) < 2:
        return 0.0

 Read the full in Notebook...
```

- ۱. `downside_diffs = np.minimum(returns_array - target_return, 0)` - فقط خسارات (منفی‌ها) انتخاب میشوند
- ۲. `downside_std` - انحراف معیار فقط برای خسارات محاسبه میشود
- اگر Sortino : `annualize=False` ماهانه حساب میشود
- اگر `annualize=True` : `trades_per_year = num_trades * 12` و `annualization_factor = sqrt(trades_per_year)`
- ۳. `downside_std` را در `annualization_factor` ضرب میکنیم تا ریسک نزولی سالانه شود
- ۴. اگر `downside_std == 0` برای اینکه خروجی بینهایت نشود یک مقدار بزرگ برمیگردانم

> وقتی معاملات کم هستند، سالانه‌سازی نتایج بزرگ‌تری می‌دهد. به خصوص اینو جلوتر میبینیم. چون تعداد ترید ها کمه یکی دو ترید در ماه انجام میدیم به خاطر همین *STD* خیلی کم میشود و مقدار شارپ بزرگ میشود

```
def calculate_time_under_water(trades, initial_capital):
    if not trades:
        return 0

 Read the full in Notebook...
```

در این فانکشن تعداد معاملاتی که پورتفولیو زیر بالاترین نقطه‌اش است (recovery time) را محاسبه میکنیم.
یعنی چند معامله طول کشید تا از یک ضرر بزرگ بهبود پیدا کنیم.

```
def calculate_profit_factor(trades):
    if not trades:
        return 0.0

 Read the full in Notebook...
```

در این فانکشن نسبت کل سودها به کل خسارت‌ها را محاسبه میکنیم. Profit Factor > 2 یعنی برای هر یک دلار خسارت، دو دلار سود داریم، که خیلی خوب است.

```
def calculate_grid_efficiency(state):
    tp_count = sum(1 for trade in state['trades'] if trade.get('reason')
== 'TP')
    sl_count = sum(1 for trade in state['trades'] if trade.get('reason')
== 'SL')

 Read the full in Notebook...
```

در این فانکشن درصد معاملاتی که به TP رسیدند را محاسبه میکنیم
فرمول: Grid Efficiency = (معاملات برنده / کل معاملات بسته شده) × 100.
هرچه بالاتر یعنی استراتژی بیشتر موفق بوده است.

```
def calculate_max_drawdown(trades, initial_capital):
    if not trades:
        return 0.0

 Read the full in Notebook...
```

در این فانکشن بزرگترین کاهش پورتفولیو از بالاترین نقطه‌اش را محاسبه میکنیم. مثلاً اگر Max Drawdown = 20% یعنی در بدترین شرایط، پورتفولیو 20% کاهش یافته است. این معیار ریسک سرمایه گذاری را نشان میدهد.

```
def get_trading_statistics(state, timeframe='1d', annualize=True):
    win_rate = (state['winning_trades'] / state['total_trades'] * 100) if
state['total_trades'] > 0 else 0
```

```
Read the full in Notebook...
```

در این فانکشن تمام متریک‌های مهم را جمع‌آوری میکنیم و در یک دیکشنری برمی‌گردانیم تا بتوانیم نتایج کامل استراتژی را تحلیل کنیم.

مقایسه ها

```python
def compare_strategies(symbol, year=2025, initial_capital=INITIAL_CAPITAL,
base_volume=BASE_VOLUME):
    timeframes = ['1h', '4h', '1d']
    comparison_results = {}

Read the full in Notebook...
```

در این فانکشن دو استراتژی (Martingale و Anti-Martingale) را برای هر تایم فریم تست میکنیم و نتایج را مقایسه میکنیم.

```
def run_complete_analysis(symbol, year=2025,
initial_capital=INITIAL_CAPITAL, base_volume=BASE_VOLUME):

Read the full in Notebook...
```

در این فانکشن تحلیل کامل انجام میدهیم: ابتدا دو استراتژی را مقایسه کرده سپس نتایج ماهانه هر کدام را چاپ میکنیم.

```
def find_qualifying_coins():
    ema_period = 200
    all_results = {}

Read the full in Notebook...
```

در این فانکشن تمام کوین‌ها را چک میکنیم که کدام‌ها شرط EMA را برای کدام تایم فریم‌ها داشتند و اونایی که در تمام تایم فریم‌ها پاس شدند را کوین‌های معتبر معرفی میکنیم. (در همان تایم فریم پاس شده استراتژی رو انجام میدیم)

```
def create_ema_results_dataframe(all_results, ema_period=EMA):
    results_list = []
    ema_coins = all_results.get(ema_period, [])

Read the full in Notebook...
```

در این فانکشن نتایج EMA را به صورت یک DataFrame درمیاریم تا بهتر بتونیم ببینیم کدوم کوین‌ها در کدام تایم فریم‌ها معتبر هستند.

```
def test_grid_on_qualifying_coins(df_coins, year=2025,
initial_capital=INITIAL_CAPITAL, base_volume=BASE_VOLUME):
    grid_results = {}


Read the full in Notebook...
```

در این فانکشن کوین‌های معتبر را تست میکنیم:
برای هر کوین و تایم فریمی که شرط رو پاس کرده، هر دو استراتژی را اجرا میکنیم و میانگین نتایج را چاپ درمیاریم.

پلات کردن

```
def plot_backtest_results(df_price, state, symbol, timeframe,
strategy_name='Strategy'):
    plt.style.use('seaborn-v0_8-darkgrid')
    fig = plt.figure(figsize=(16, 12), facecolor='#f8f9fa')


Read the full in Notebook...
```

در این پلات ما سه نمودار داریم:

- نمودار اول: قیمت کوین به همراه سطوح گرید
- نمودار دوم: Equity Curve
- نمودار سوم: سود/زیان هر معامله به صورت جداگانه

و برای هر کوین و هرکدام دو استراتژی این پلات رو درست میکنیم.

```
def plot_strategy_comparison(comparison_results, symbol, year=2025):
    timeframes = ['1h', '4h', '1d']
    metrics = ['avg_profit', 'win_rate', 'return_pct', 'sharpe_ratio',
'max_drawdown']
    metric_labels = ['Avg Profit ($)', 'Win Rate (%)', 'Return (%)',
'Sharpe Ratio', 'Max DD (%)']

Read the full in Notebook...
```

در این پلات ما دو استراتژی (Martingale و Anti-Martingale) را برای هر سه تایم‌فریم (1H, 4H, 1D) مقایسه میکنیم:

- در سطر اول: سود متوسط و درصد بازدهی برای هر تایم‌فریم
- در سطر دوم: معیارهای ریسک (Win Rate، Sharpe Ratio، Max Drawdown) برای هر تایم‌فریم
- رنگ آبی نمایانگر Martingale و نارنجی نمایانگر Anti-Martingale است

```
def plot_monthly_equity_curves(monthly_results, symbol, timeframe,
strategy_name='Strategy'):
    if not monthly_results:
        return None

  Read the full in Notebook...
```

در این فانکشن برای هر ماه یک شاخص نمایش میدهیم (حداکثر ۱۲ ماه در ۴ ستون):

- هر شاخص شامل: نام ماه (سال-ماه)، تعداد معاملات، Win Rate، سود/زیان، Sharpe Ratio، Return %، Max Drawdown
- رنگ پس‌زمینه: سبز اگر سود باشد، قرمز اگر ضرر باشد

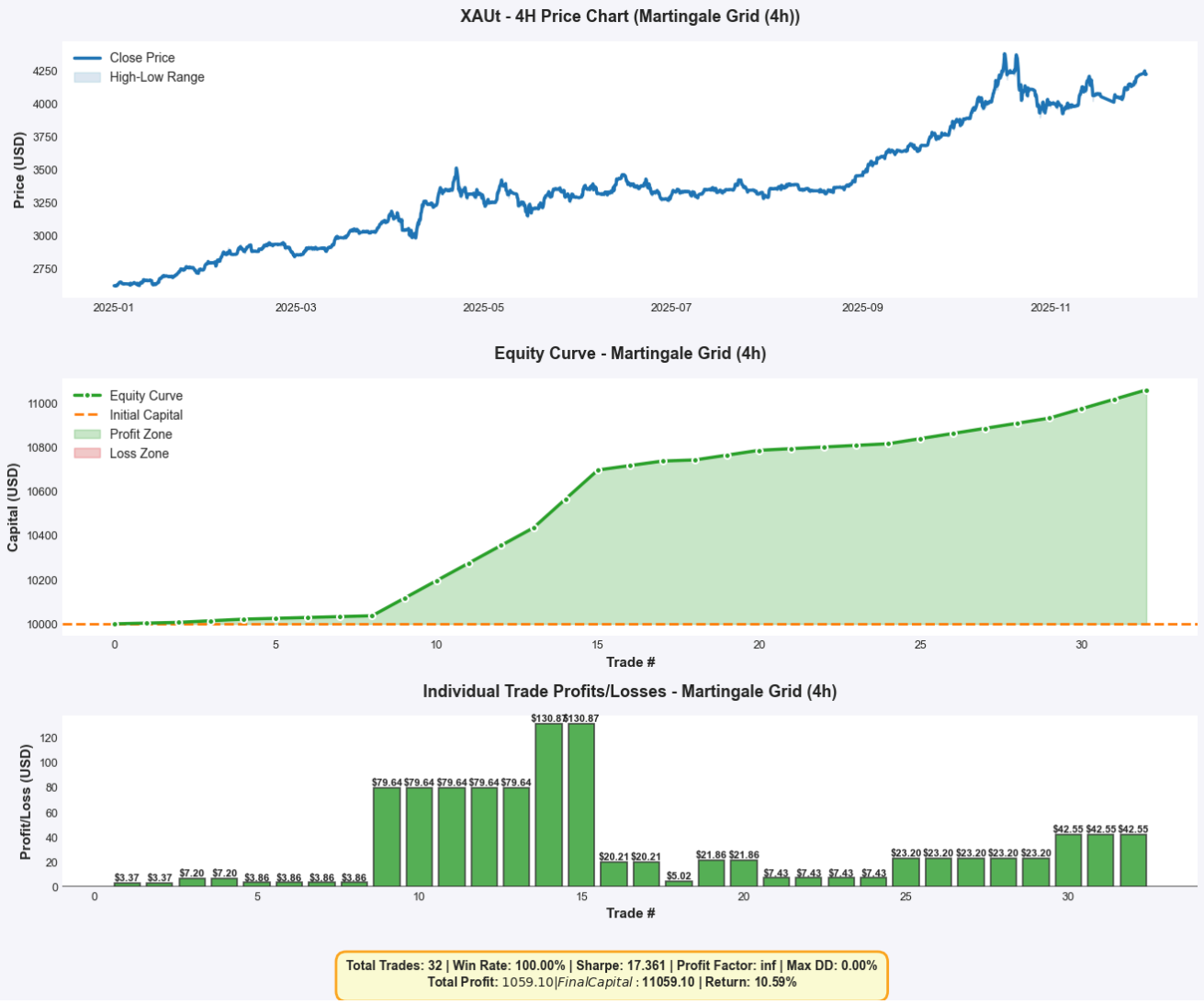
نتایج گرفته شده

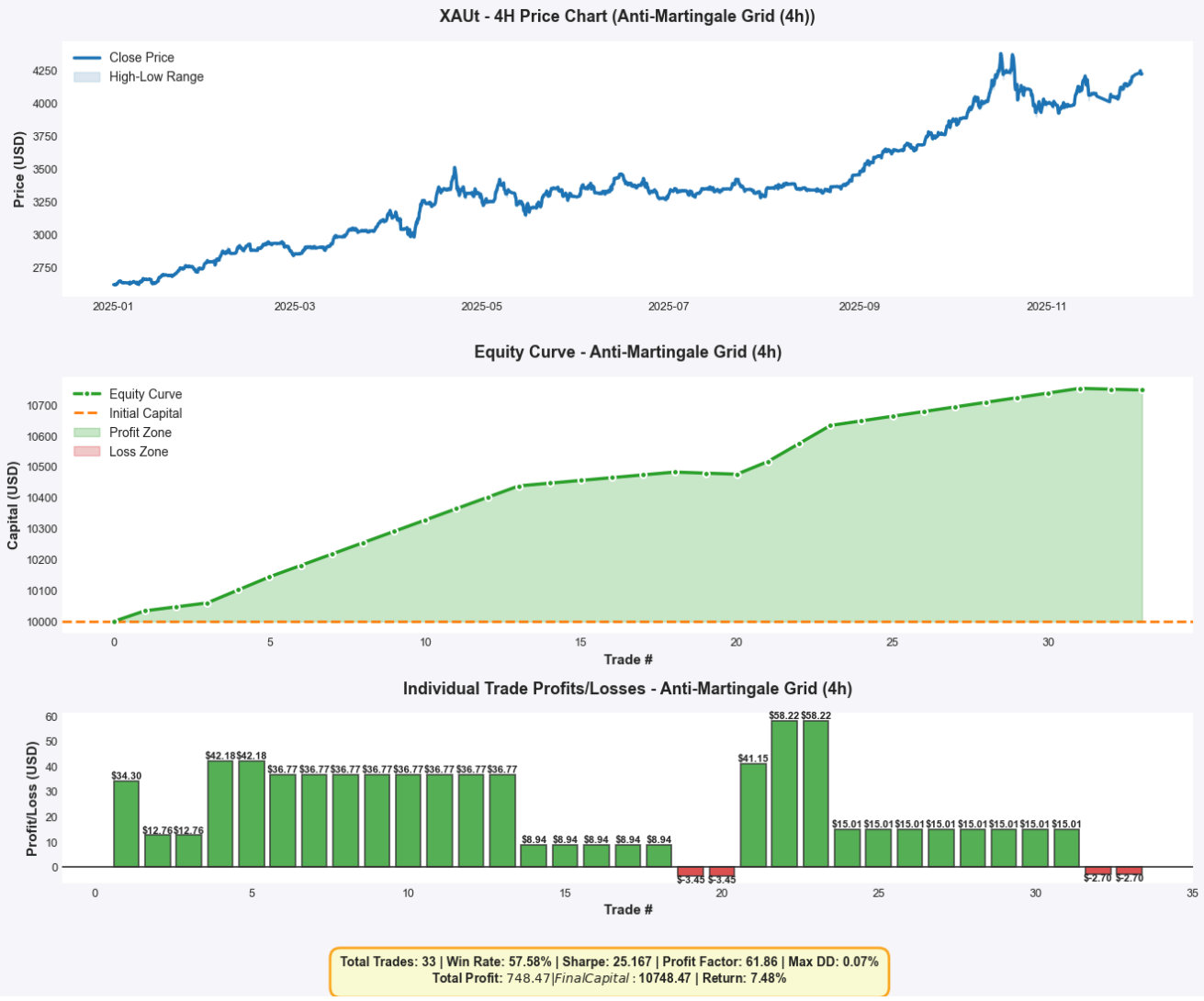
درآخر در کد زیر کوین هارو پیدا و دو استراتژی رو تست میکنیم و متریک هارا به دست می آوریم:

```
qualifying_coins, all_results = find_qualifying_coins()

df_ema_200 = create_ema_results_dataframe(all_results, ema_period=EMA)

Read the full in notebook...
```

> چون تعداد نتایج بالا بود و برای شلوغ نشدن داک من فقط به توضیح سه مورد بسنده میکنم (کل نمودارها در نوتبوک قابل مشاهده هستند):

XAUt-4h


XAUt - 4H Price Chart (Martingale Grid (4h))
Close Price
High-Low Range
Price (USD)
4250
4000
3750
3500
3250
3000
2750
2025-01
2025-03
2025-05
2025-07
2025-09
2025-11
Equity Curve - Martingale Grid (4h)
Equity Curve
Initial Capital
Profit Zone
Loss Zone
Capital (USD)
11000
10800
10600
10400
10200
10000
0
5
10
15
20
25
30
Trade #
Individual Trade Profits/Losses - Martingale Grid (4h)
Profit/Loss (USD)
120
100
80
60
40
20
0
$3.37 $3.37 $7.20 $7.20 $3.86 $3.86 $3.86 $3.86 $79.64 $79.64 $79.64 $79.64 $79.64 $130.87 $130.87 $20.21 $20.21 $5.02 $21.86 $21.86 $7.43 $7.43 $7.43 $7.43 $23.20 $23.20 $23.20 $23.20 $23.20 $42.55 $42.55 $42.55
0
5
10
15
20
25
30
Trade #
Total Trades: 32 | Win Rate: 100.00% | Sharpe: 17.361 | Profit Factor: inf | Max DD: 0.00%
Total Profit: 1059.10|FinalCapital : 11059.10 | Return: 10.59%



Total Trades: 33 | Win Rate: 57.58% | Sharpe: 25.167 | Profit Factor: 61.86 | Max DD: 0.07%
Total Profit: 748.47 | *FinalCapital* : **10748.47 | Return: 7.48%**

XAUt - 4H - Monthly Equity Curves (Martingale Grid)

Month: 2025-01 Trades: 2 Win Rate: 100.00% Profit: $6.74 Return: 0.07% Sharpe: 1119.552 Max DD: 0.00%	**Month: 2025-02** Trades: 2 Win Rate: 100.00% Profit: $14.41 Return: 0.14% Sharpe: 72.995 Max DD: 0.00%	**Month: 2025-03** Trades: 4 Win Rate: 100.00% Profit: $15.45 Return: 0.15% Sharpe: 53.301 Max DD: 0.00%	**Month: 2025-04** Trades: 5 Win Rate: 100.00% Profit: $398.19 Return: 3.98% Sharpe: 6.160 Max DD: 0.00%
Month: 2025-05 Trades: 2 Win Rate: 100.00% Profit: $261.75 Return: 2.62% Sharpe: 9.533 Max DD: 0.00%	**Month: 2025-06** Trades: 2 Win Rate: 100.00% Profit: $40.42 Return: 0.40% Sharpe: 6.666 Max DD: 0.00%	**Month: 2025-07** Trades: 1 Win Rate: 100.00% Profit: $5.02 Return: 0.05% Sharpe: 0.000 Max DD: 0.00%	**Month: 2025-08** Trades: 2 Win Rate: 100.00% Profit: $43.72 Return: 0.44% Sharpe: 5.791 Max DD: 0.00%
Month: 2025-09 Trades: 4 Win Rate: 100.00% Profit: $29.73 Return: 0.30% Sharpe: 8.190 Max DD: 0.00%	**Month: 2025-10** Trades: 5 Win Rate: 100.00% Profit: $116.02 Return: 1.16% Sharpe: 15.528 Max DD: 0.00%	**Month: 2025-11** Trades: 3 Win Rate: 100.00% Profit: $127.65 Return: 1.28% Sharpe: 11.083 Max DD: 0.00%	**Month: 2025-12** Trades: 0 Win Rate: 0.00% Profit: $0.00 Return: 0.00% Sharpe: 0.000 Max DD: 0.00%

XAUt - 4H - Monthly Equity Curves (Anti-Martingale Grid)

Month: 2025-01 Trades: 1 Win Rate: 100.00% Profit: $34.30 Return: 0.34% Sharpe: 0.000 Max DD: 0.00%	**Month: 2025-02** Trades: 2 Win Rate: 50.00% Profit: $25.52 Return: 0.26% Sharpe: 0.001 Max DD: 0.13%	**Month: 2025-03** Trades: 2 Win Rate: 100.00% Profit: $84.37 Return: 0.84% Sharpe: 14.349 Max DD: 0.00%	**Month: 2025-04** Trades: 8 Win Rate: 50.00% Profit: $294.16 Return: 2.94% Sharpe: -0.030 Max DD: 1.71%
Month: 2025-05 Trades: 5 Win Rate: 40.00% Profit: $44.72 Return: 0.45% Sharpe: -0.274 Max DD: 0.44%	**Month: 2025-06** Trades: 2 Win Rate: 50.00% Profit: $-6.90 Return: -0.07% Sharpe: -0.113 Max DD: 0.43%	**Month: 2025-07** Trades: 0 Win Rate: 0.00% Profit: $0.00 Return: 0.00% Sharpe: 0.000 Max DD: 0.00%	**Month: 2025-08** Trades: 1 Win Rate: 100.00% Profit: $41.15 Return: 0.41% Sharpe: 0.000 Max DD: 0.00%
Month: 2025-09 Trades: 2 Win Rate: 100.00% Profit: $116.45 Return: 1.16% Sharpe: 23.358 Max DD: 0.00%	**Month: 2025-10** Trades: 8 Win Rate: 50.00% Profit: $120.10 Return: 1.20% Sharpe: -0.104 Max DD: 1.64%	**Month: 2025-11** Trades: 2 Win Rate: 50.00% Profit: $-5.39 Return: -0.05% Sharpe: -0.089 Max DD: 0.51%	**Month: 2025-12** Trades: 0 Win Rate: 0.00% Profit: $0.00 Return: 0.00% Sharpe: 0.000 Max DD: 0.00%

همان طور که میبینیم برای این کوین خوب هردو استراتژی خوب جواب میدهند.
و اگر بخواهیم دقیق تر متریک هارا برای کل بازه داشته باشیم:

```
XAUt - 4H
--------------------------------------------------------------------------------
------

Martingale Grid Statistics:
  Total Profit: $1,059.10
  Final Capital: $11,059.10
  Return: 10.59%
  Total Trades: 32
  Exits with TP: 32 | Exits with SL: 0
  Winning Trades: 32
  Losing Trades: 0
  Win Rate: 100.00%
  Sharpe Ratio: 17.3615
  Sortino Ratio: 999.9900 (no losing trade)
  Max Drawdown: 0.00%
  Time Under Water: 0.00 days
  Profit Factor: inf (No losing trades)
  Grid Efficiency: inf (All trades hit TP)

Anti-Martingale Grid Statistics:
  Total Profit: $748.47
  Final Capital: $10,748.47
  Return: 7.48%
  Total Trades: 33
  Exits with TP: 29 | Exits with SL: 4
  Winning Trades: 19
  Losing Trades: 14
  Win Rate: 57.58%
  Sharpe Ratio: 25.1665
  Sortino Ratio: 404.5670
  Max Drawdown: 0.07%
  Time Under Water: 4.00 days
  Profit Factor: 61.8647
  Grid Efficiency: 87.8788
```

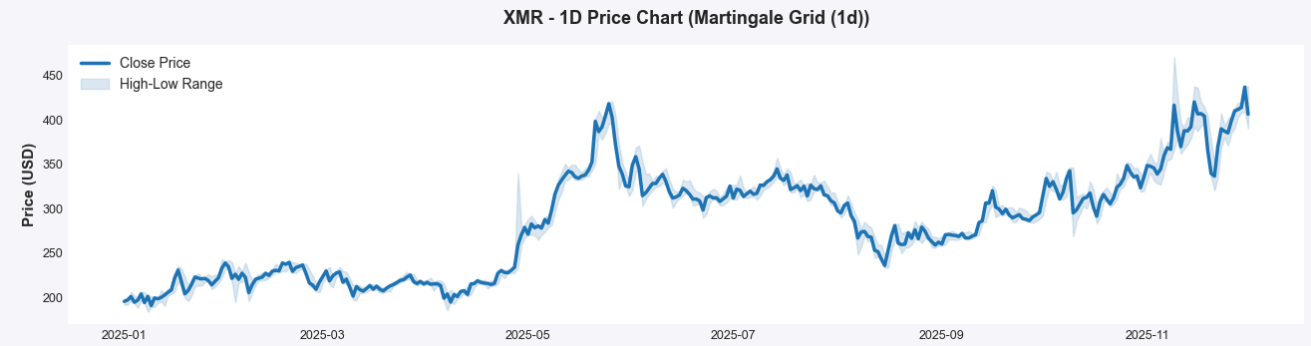
USDT - 1d

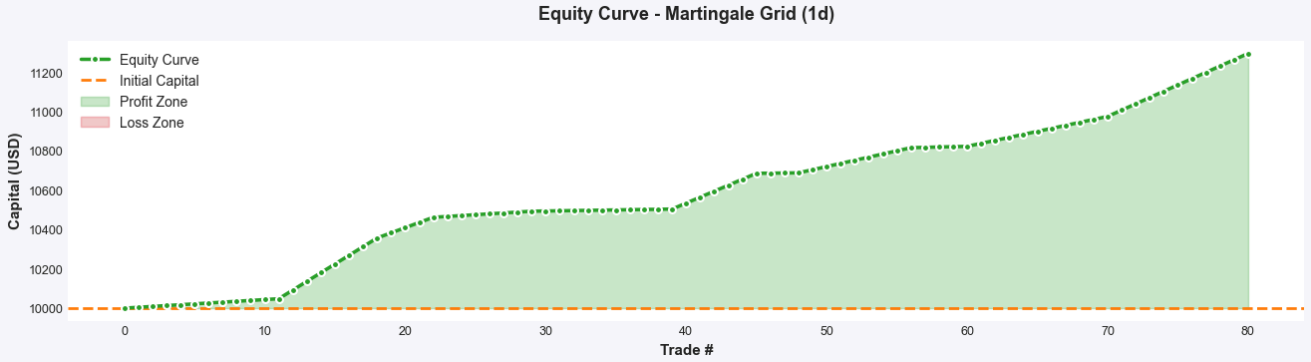
ولی همه کوین ها به دست آمده لزوما برای ترید مناسب نیستند.
به عنوان مثال تتر که شرط سوال رو پاس کرده است. ولی اگر به نمودار قیمت آن نگاه کنیم میبینیم که به شدت تغییرات کمی در قیمت خود داشته است. و فاصله گریدهای ما برای این کوین بزرگ بوده و اگر بخواهیم حتما روی این کوین استراتژی هارا انجام بدهیم باید فاصله را خیلی کمتر کنیم.

USDT - 1D Price Chart (Martingale Grid (1d))
Close Price
High-Low Range
Price (USD)
1.008
1.006
1.004
1.002
1.000
0.998
2025-01
2025-03
2025-05
2025-07
2025-09
2025-11
Equity Curve - Martingale Grid (1d)
Equity Curve
Initial Capital
Profit Zone
Loss Zone
Capital (USD)
10400
10200
10000
9800
9600
0.0
0.2
0.4
0.6
0.8
1.0
Trade #
Trade Results
No detailed trades (aggregated view)
Total Trades: 0 | Win Rate: 0.00% | Sharpe: 0.000 | Profit Factor: 0.00 | Max DD: 0.00%
Total Profit: 0.00|FinalCapital: 10000.00 | Return: 0.00%

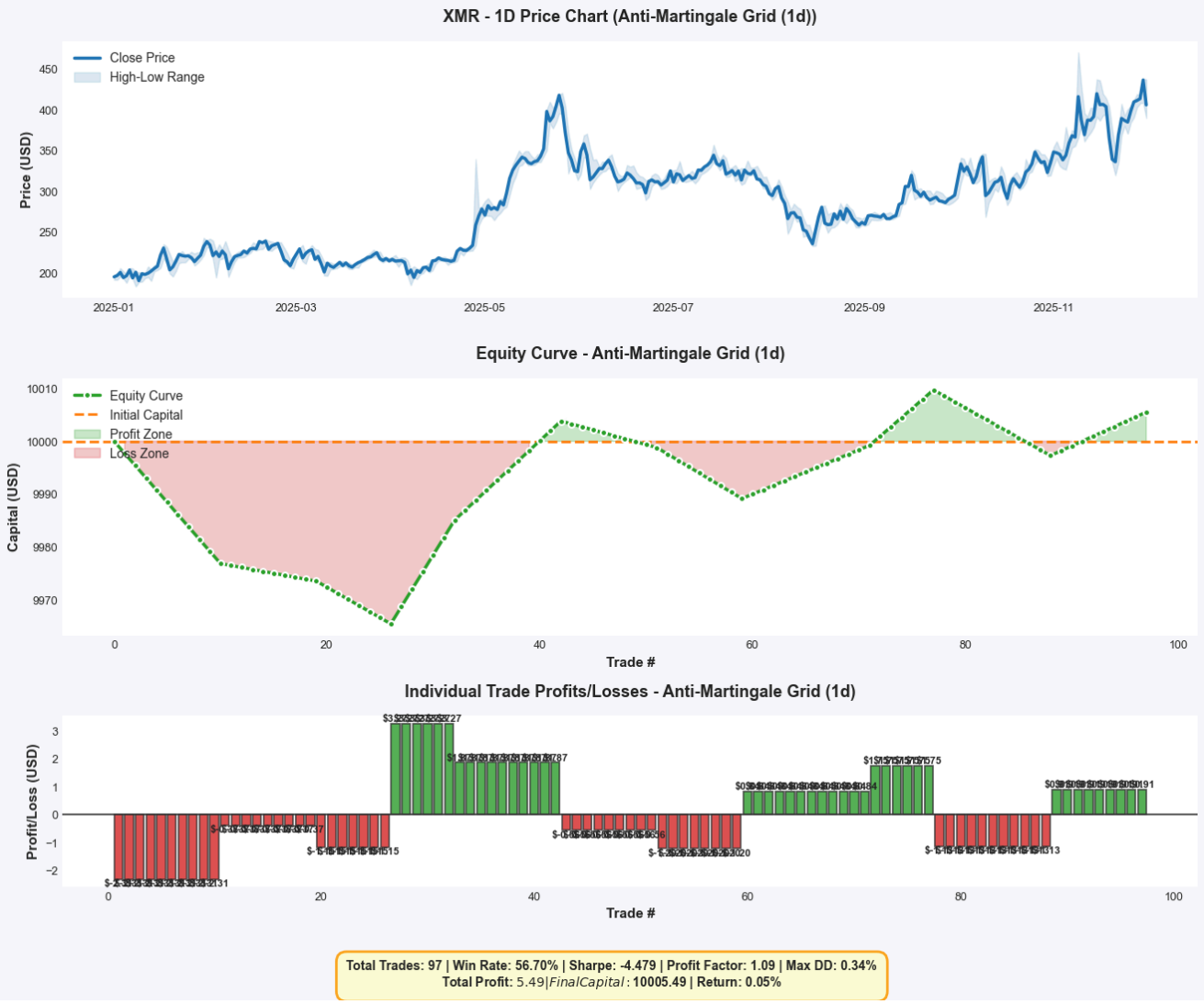
XMR - 1d

همان طور که در نمودارهای زیر مشخص است استراتژی *Martingale* برای این کوین خوب عمل کرده است. اما *Anti martingale* درکل سودده ولی نوسان های زیادی داشته است.





Total Trades: 80 | Win Rate: 100.00% | Sharpe: 34.660 | Profit Factor: inf | Max DD: 0.00%
Total Profit: 1297.86|*FinalCapital* : **11297.86 | Return: 12.98%**



XMR - 1D - Monthly Equity Curves (Martingale Grid)

Month: 2025-01
Trades: 11
Win Rate: 100.00%
Profit: $48.02
Return: 0.48%
Sharpe: 2.898
Max DD: 0.00%

Month: 2025-02
Trades: 7
Win Rate: 100.00%
Profit: $309.50
Return: 3.09%
Sharpe: 1.876
Max DD: 0.00%

Month: 2025-03
Trades: 4
Win Rate: 100.00%
Profit: $105.36
Return: 1.05%
Sharpe: 2.695
Max DD: 0.00%

Month: 2025-04
Trades: 7
Win Rate: 100.00%
Profit: $31.07
Return: 0.31%
Sharpe: 1.954
Max DD: 0.00%

Month: 2025-05
Trades: 10
Win Rate: 100.00%
Profit: $10.96
Return: 0.11%
Sharpe: 1.615
Max DD: 0.00%

Month: 2025-06
Trades: 6
Win Rate: 100.00%
Profit: $182.18
Return: 1.82%
Sharpe: 3.595
Max DD: 0.00%

Month: 2025-07
Trades: 3
Win Rate: 100.00%
Profit: $2.44
Return: 0.02%
Sharpe: 30.690
Max DD: 0.00%

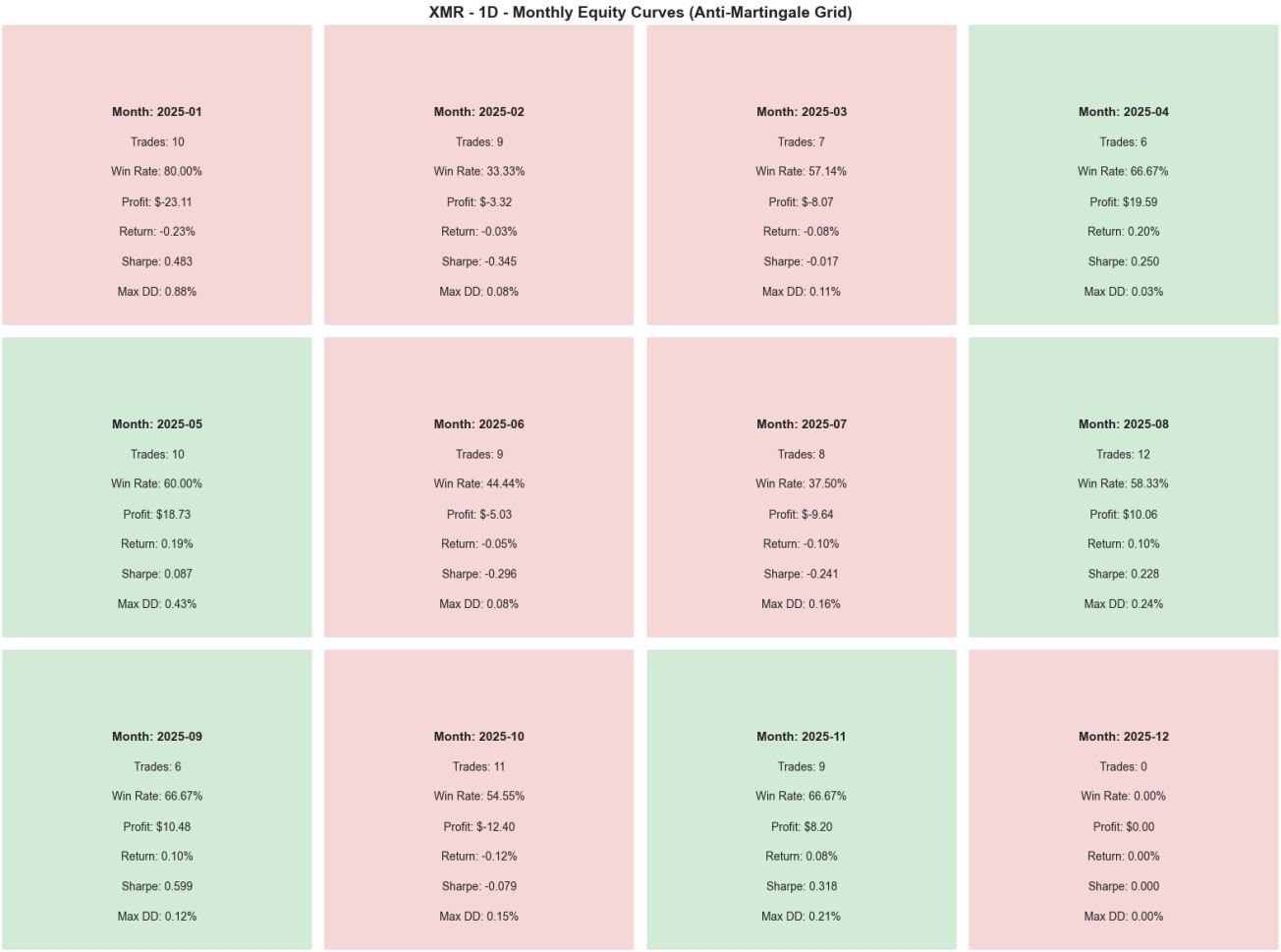
Month: 2025-08
Trades: 8
Win Rate: 100.00%
Profit: $128.19
Return: 1.28%
Sharpe: 3.390
Max DD: 0.00%

Month: 2025-09
Trades: 4
Win Rate: 100.00%
Profit: $6.87
Return: 0.07%
Sharpe: 3.605
Max DD: 0.00%

Month: 2025-10
Trades: 10
Win Rate: 100.00%
Profit: $152.39
Return: 1.52%
Sharpe: 2.542
Max DD: 0.00%

Month: 2025-11
Trades: 10
Win Rate: 100.00%
Profit: $320.89
Return: 3.21%
Sharpe: 1.580
Max DD: 0.00%

Month: 2025-12
Trades: 0
Win Rate: 0.00%
Profit: $0.00
Return: 0.00%
Sharpe: 0.000
Max DD: 0.00%



چرا Anti-Martingale بدتر عمل کرد؟

۱. الگوی متناوب سود/زیان (Whipsaw Pattern)

- ماه‌های متناوب بین سود و زیان: Jan(−), Feb(−), Mar(−), Apr(+), May(+), Jun(−), Jul(−), Aug(+), ...
- این الگوی متناوب مشکل است چون:
  - بعد از یک سودده معامله، موقعیت بزرگتر باز می‌شود
  - اما بازار معکوس می‌شود و معامله بعدی ضرر می‌دهد
  - سپس حجم به پایه برمی‌گردد و دوباره شروع می‌شود

۲. عدم استراتژی برای بازار نوسانی

- Martingale: بعد از زیان، حجم افزایش می‌یابد → خریدهای بیشتر در قیمت‌های پایین‌تر
- Anti-Martingale: بعد از سود، حجم افزایش می‌یابد → اما اگر ترند معکوس شود، ضرر بزرگتر است
- در مواقعی که قیمت بالا رفته بود، حجم افزایش داد Anti-Martingale نوسانات زیادی داشت، بنابراین XMR و سپس سقوط شدید رخ داد

۳. عدم تطابق با رفتار قیمت XMR

- Martingale برای بازارهای نزولی پایدار بهتر است (منتظر صعود)
- Anti-Martingale برای بازارهای صعودی پایدار بهتر است (به پیروی از ترند)
- XMR رفتار نوسانی داشت، نه صعودی و نه نزولی پایدار

درآخر متریک های این کوین هم به صورت زیر میباشد:

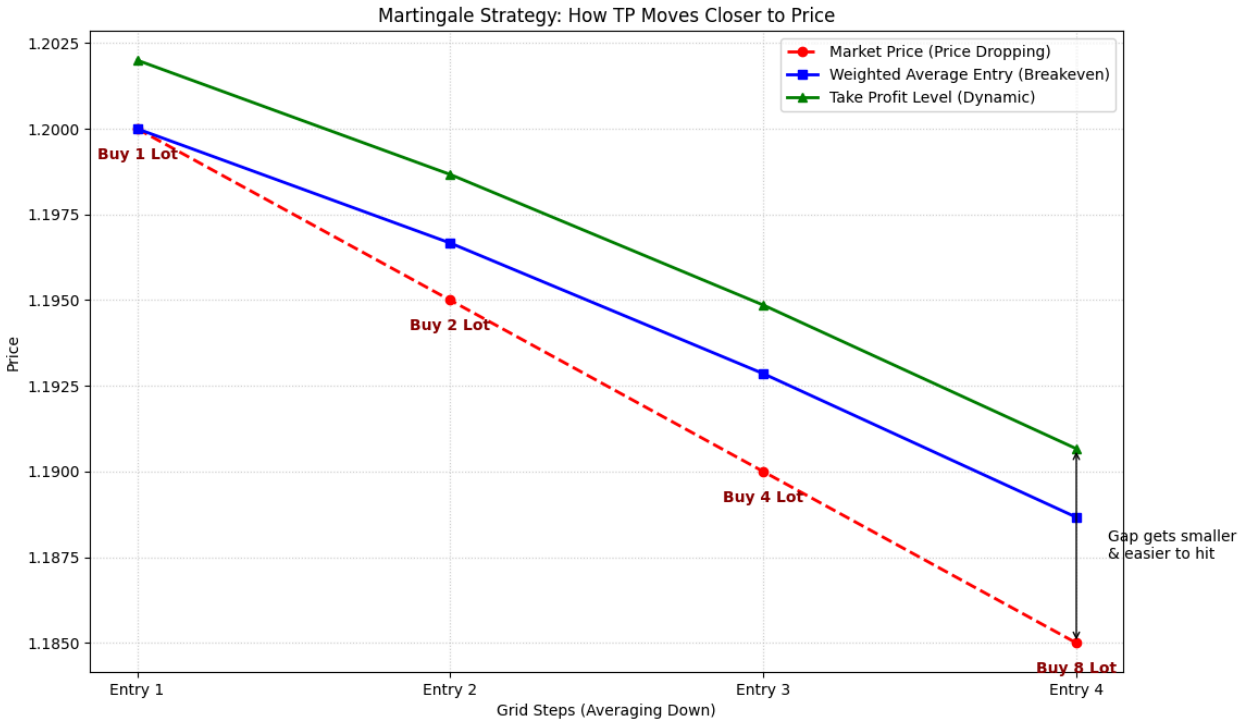
```
XMR - 1D
--------------------------------------------------------------------------------
------

Martingale Grid Statistics:
  Total Profit: $1,297.86
  Final Capital: $11,297.86
  Return: 12.98%
  Total Trades: 80
  Exits with TP: 80 | Exits with SL: 0
  Winning Trades: 80
  Losing Trades: 0
  Win Rate: 100.00%
  Sharpe Ratio: 34.6601
  Sortino Ratio: 999.9900 (no losing trade)
  Max Drawdown: 0.00%
  Time Under Water: 0.00 days
  Profit Factor: inf (No losing trades)
  Grid Efficiency: inf (All trades hit TP)

Anti-Martingale Grid Statistics:
  Total Profit: $5.49
  Final Capital: $10,005.49
  Return: 0.05%
  Total Trades: 97
  Exits with TP: 43 | Exits with SL: 54
  Winning Trades: 55
  Losing Trades: 42
  Win Rate: 56.70%
  Sharpe Ratio: -4.4788
  Sortino Ratio: -7.0325
  Max Drawdown: 0.34%
  Time Under Water: 90.00 days
  Profit Factor: 1.0893
  Grid Efficiency: 44.3299
```

بخش ششم

سوال اول: چرا در استراتژی Martingale Grid ، حتی با وجود ریزش قیمت نقطه خروج (Take Profit) به قیمت بازار نزدیک تر میشود؟ (با رسم نمودار توضیح دهید.)

در استراتژی مارتینگل گرید، دلیل نزدیک شدن نقطه خروج (Take Profit) به قیمت بازار، مکانیزم میانگین‌گیری وزنی میباشد.
زمانی که قیمت خلاف جهت معامله حرکت می‌کند، ما پله‌های خرید بعدی را با حجمی چند برابر (مثلا دو برابر پله قبلی) باز می‌کنیم. این حجم سنگین در قیمت‌های پایین همانند یک وزنه بزرگ عمل میکنه و میانگین قیمت ورود کل را با شدت زیادی به سمت پایین می‌کشه. در نتیجه، نقطه خروج که همواره فاصله ثابتی با این میانگین دارد، به

شدت افت میکنه و به قیمت فعلی بازار نزدیک می‌شود، به طوری که تنها با یک اصلاح قیمت کوچک و نه بازگشت کامل روند، کل مجموعه پوزیشن‌ها با سود بسته می‌شوند.



فرمول میانگین وزنی:

$$\text{Avg Price} = \frac{\sum_{i=0}^{n}(\text{Price}_i \times \text{Volume}_i)}{\sum_{i=0}^{n}\text{Volume}_i} = \frac{\sum_{i=0}^{n}(\text{Price}_i \times \text{BaseLot} \times 2^i)}{\text{BaseLot} \times (2^{n+1} - 1)}$$

همانطور که مشخص است، پله‌های پایینی (با حجم نمایی) وزن بسیار بیشتری دارند و میانگین را به شدت پایین می‌کشند.

سوال دوم : خطرات اصلی استفاده از شبکه های مارتینگل در بازارهای(Bear Market) طولانی مدت چیست؟

خطرات اصلی استفاده از شبکه مارتینگل در بازارهای نزولی طولانی‌مدت (Bear Market) عبارتند از:

۱. رشد نمایی حجم و کال مارجین (Margin Call):

خطرناک‌ترین ریسک، تصاعد هندسی حجم معاملات (2^n) است. در یک روند نزولی طولانی بدون اصلاح، تعداد پله‌های خرید زیاد می‌شود و حجم معاملات به قدری بالا می‌رود که Free Margin حساب تمام میشه و کارگزاری تمام پوزیشن‌ها را با ضرر می‌بندد. (لیکوییید شدن)

$$\text{Total Volume} = \sum_{i=0}^{n} \text{BaseLot} \times 2^i = \text{BaseLot} \times (2^{n+1} - 1)$$

۲. عدم بازگشت قیمت به میانگین (No Mean Reversion):

مارتینگل بر این فرض استوار است که قیمت نهایتا اصلاح خواهد کرد. در یک بازار Bear قوی، قیمت می‌تواند ماه‌ها بدون هیچ اصلاح قابل‌توجهی فقط بریزد. در این حالت، میانگین قیمت ورودی شما هرگز لمس نمی‌شود.

۳. ضرر شناور غیرقابل کنترل (Deep Drawdown):

حتی اگر حساب لیکویید نشود، بخش عظیمی از سرمایه به عنوان ضرر شناور درگیر می‌شود. این باعث افت شدید Equity می‌شود و عملا حساب را فلج می‌کند.

۴. ریسک صفر شدن دارایی (مخصوصا کریپتو/سهام):

برخلاف جفت‌ارزها، در بازار کریپتو یا سهام، یک دارایی ممکن است در بازار Bear به سمت ارزش صفر میل کند (مانند LUNA). میانگین کم کردن روی دارایی که ارزش ذاتی خود را از دست داده، سریع‌ترین راه برای نابودی کامل سرمایه است.

سوال سوم : تفاوت اصلی بین یک Grid ساده با حجم ثابت و یک Martingale Grid را از نظر مدیریت سرمایه شرح دهید.

تفاوت های اصلی:

۱. نحوه رشد ریسک (خطی در برابر نمایی):

در Grid ساده، حجم پوزیشن‌ها ثابت هست. بنابراین ریسک و مارجین درگیر به صورت خطی افزایش می‌یابد ($1 + 1 + 1 \ldots$).
این روش به ما اجازه می‌دهد ریزش‌های بسیار عمیق‌تری را با سرمایه ثابت تحمل کنیم.
اما در Martingale، ریسک به صورت نمایی ($1 + 2 + 4 \ldots$) رشد می‌کند. به طوری که باز شدن چند پله آخر می‌تواند کل مارجین حساب را در لحظه ببلعد و منجر به کال مارجین شود.

مثال عددی برای رشد ریسک:

- ساده (10 پله): $1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 = \$10$ سرمایه درگیر Grid
- Martingale (10 پله): $1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512 = \$1,023$ سرمایه درگیر

همانطور که مشخص است، در Martingale پله دهم به تنهایی از کل 9 پله قبلی بزرگتر است!

۲. نقطه سربه‌سر (Breakeven):

در Grid ساده، میانگین قیمت خرید تقریبا در وسط دامنه نوسان قرار می‌گیرد. بنابراین برای خروج با سود، بازار باید اصلاح عمیقی (حدود ۵۰٪ مسیر ریزش) انجام دهد که ممکن است ماه‌ها طول بکشد (خطر خواب سرمایه).
اما در Martingale، میانگین قیمت به شدت به قیمت جاری بازار نزدیک می‌شود و تنها با یک اصلاح جزئی، کل سبد وارد سود می‌شود (خطر نابودی حساب در عوض خروج سریع).

۳. مدیریت سرمایه و تخصیص منابع:

در **Grid ساده**، سرمایه به صورت یکنواخت و کاملا قابل پیش‌بینی توزیع می‌شود.
از آنجا که سایز پوزیشن‌ها ثابت است، ما از قبل دقیقا می‌دانیم که برای پوشش مثلا ۵۰ پله ریزش، به چه میزان مارجین نیاز دارید و می‌توانیم حد ضرر (Stop Loss) را دقیق محاسبه کنید.

اما در **Martingale**، تخصیص سرمایه به شدت تهاجمی و متمرکز در قیمت‌های پایین است.
بخش عمده‌ی سرمایه ما نه در شروع معامله، بلکه در بدترین شرایط بازار (کف‌های قیمتی) درگیر می‌شود.
مدیریت سرمایه در اینجا بسیار دشوار است، زیرا باز شدن ناخواسته تنها یک پله‌ی اضافه در انتهای کار (مثلا پله هفتم)، ناگهان حجمی از سرمایه را می‌طلبد که ممکن است از مجموع تمام پله‌های قبلی بیشتر باشد و کل محاسبات مدیریت ریسک ما را به هم بزند.

۴. نسبت ریسک به بازده (Risk/Reward):

در **Grid ساده**، رابطه ریسک و پاداش متعادل است. از آنجا که ریسک خطی است، ما در معرض خطر نابودی ناگهانی نیستیم، اما بهای این امنیت، صبر زیاد است.
بازدهی در این روش نیازمند زمان و نوسانات بزرگ بازار است و سرمایه ممکن است مدت‌ها درگیر بماند تا بازار به اندازه کافی برگردد.

در مقابل، **Martingale** استراتژی ریسک حداکثری برای خروج سریع است.
ما عملا ریسک نابودی کل حساب را می‌پذیریم تا در عوض، سرعت نقد شدن پوزیشن‌ها را بالا ببریم.
در این روش پاداش اصلی سود زیاد نیست، بلکه زمان کوتاه ماندن در ضرر است. یعنی با پذیرش یک ریسک خیلی بزرگ، تلاش می‌کنیم با کوچکترین اصلاح قیمت، پرونده‌ی معاملات را ببندیم.

۵. حداکثر تعداد پله قابل تحمل:

با فرض سرمایه $10,000 و قیمت $100:

Grid (حجم 1) ساده:

$$\text{Max Levels} = \frac{10,000}{100 \times 1} = 100$$

Martingale (حجم پایه 1):

$$2^{n+1} - 1 \leq \frac{10,000}{100} \implies n \leq 6$$

یعنی با همان سرمایه، Grid ساده می‌تواند 100 پله را تحمل کند، اما Martingale فقط 6-7 پله.